

Easy to Write, Fast to Build.

Operator Sums to MPOs with OpSum.jl

Lukas Devos
Flatiron Institute - CCQ

Tensor Network Operators.

Convenient language for humans:

$$H = \sum_i^K c_i \cdot O_1^{(i)} \otimes \dots \otimes O_N^{(i)}$$

- **Sum** of local operators
- Inactive sites interpreted as identity

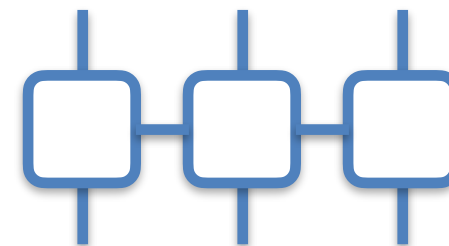
Example: Heisenberg chain

$$H = J \sum_i \vec{S}_i \cdot \vec{S}_{i+1}$$

Convenient language for tensor networks:

$$H = \prod_i^L W_i$$

- **Product** of local operators
- Single* term in the expansion



Example: Heisenberg chain

$$W_i = \begin{pmatrix} 1 & J\vec{S} & \cdot \\ \cdot & \cdot & \vec{S} \\ \cdot & \cdot & 1 \end{pmatrix}$$

Design Board

Interface and API

What is convenient/intuitive?

What do we want to support?

How do we leave room for future work?

How (much) do we hide internals?

How do we avoid sacrificing performance?

Algorithms

How do we go from a sum to a product?

What properties do we want from the output?

What is the fastest way to get there?

Extensions

Can we handle fermions?

Can we handle symmetries?

Can we handle infinite systems?

Can we handle long-range interactions?

Algorithms

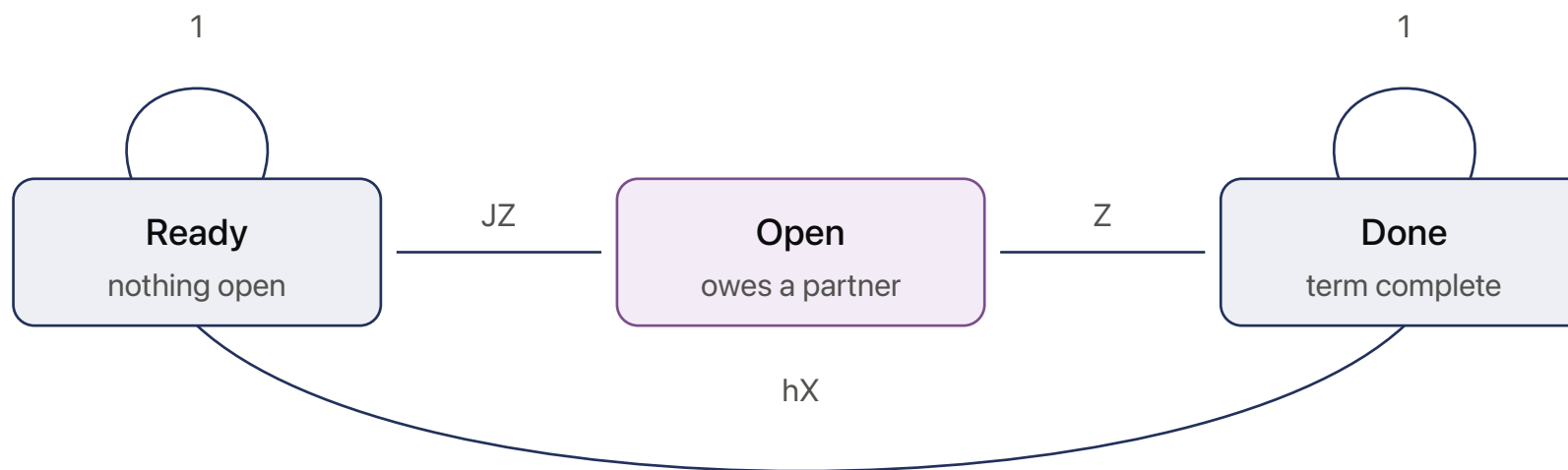
From Sums to Products

- Multiplying the W_i sums over all bond indices
 $\implies$ bond index can be interpreted as internal state
- Total is the sum of all paths from left to right boundary
- Matrix size counts how many independent states are required
- MPO construction is finding a machine whose paths generate the terms

From Sums to Products

Nearest-neighbor Transverse-field Ising model:

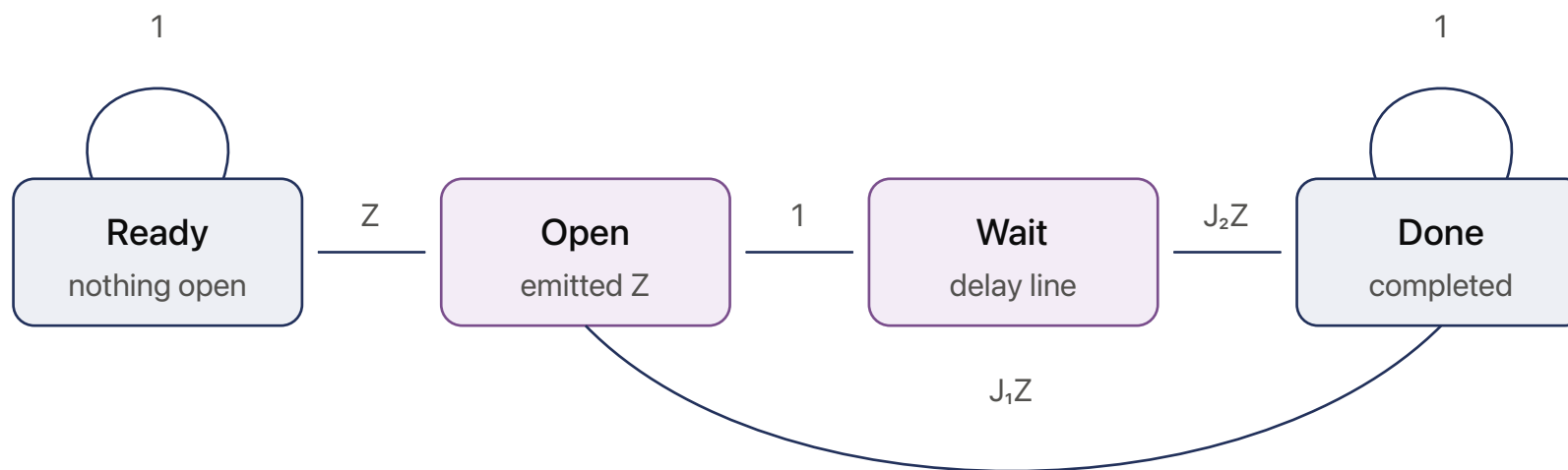
$$H = \sum_i \left(J Z_i Z_{i+1} + h X_i \right), \quad W_i = \begin{pmatrix} 1 & JZ & hX \\ \cdot & \cdot & Z \\ \cdot & \cdot & 1 \end{pmatrix}$$



From Sums to Products

Next-nearest-neighbor Transverse-field Ising model:

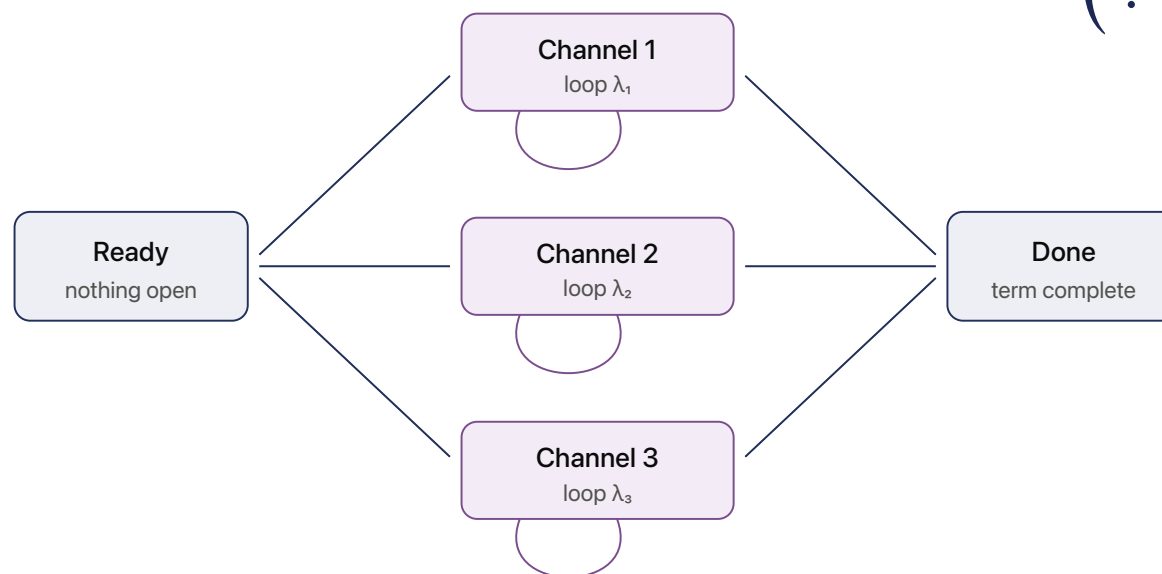
$$H = \sum_i \left(J_1 Z_i Z_{i+1} + J_2 Z_i Z_{i+2} \right), \quad W_i = \begin{pmatrix} 1 & Z & \cdot & \cdot \\ \cdot & \cdot & 1 & J_1 Z \\ \cdot & \cdot & \cdot & J_2 Z \\ \cdot & \cdot & \cdot & 1 \end{pmatrix}$$



From Sums to Products

Exponentially decaying interactions:

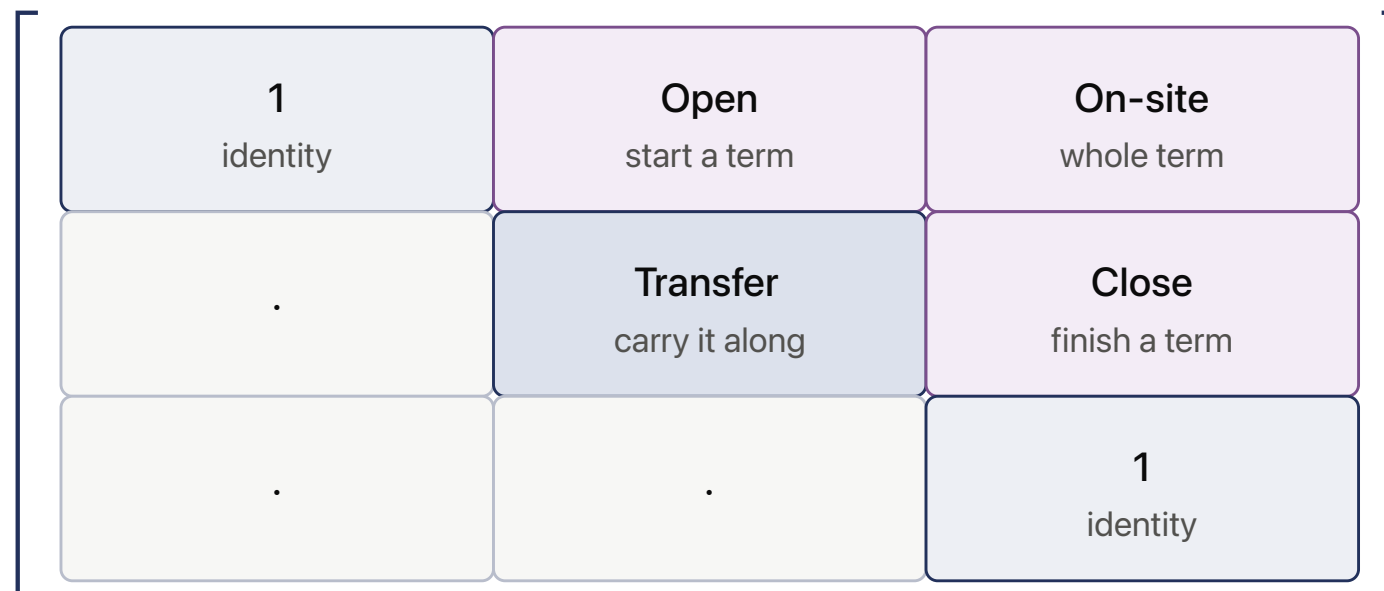
$$H = \sum_{i < j} f(j - i) O_i O_j, \quad f(r) \simeq \sum_{k=1}^m c_k \lambda_k^{r-1}, \quad W_i = \begin{pmatrix} 1 & O & O & O & \cdot \\ \cdot & \lambda_1 & \cdot & \cdot & c_1 O \\ \cdot & \cdot & \lambda_2 & \cdot & c_2 O \\ \cdot & \cdot & \cdot & \lambda_3 & c_3 O \\ \cdot & \cdot & \cdot & \cdot & 1 \end{pmatrix}$$



Desirable MPO Properties

- Exact representation: reproduces the original term term-by-term
- Small bond dimension: DMRG/TDVP sweeps become cheaper
- Structured/sparse: exact zeros and identities can be exploited in DMRG/TDVP
- Cheap to build: construction must not dominate DMRG runs

$$W_i = \begin{pmatrix} 1 & \vec{C}_i & D_i \\ 0 & A_i & \vec{B}_i \\ 0 & 0 & 1 \end{pmatrix} :$$



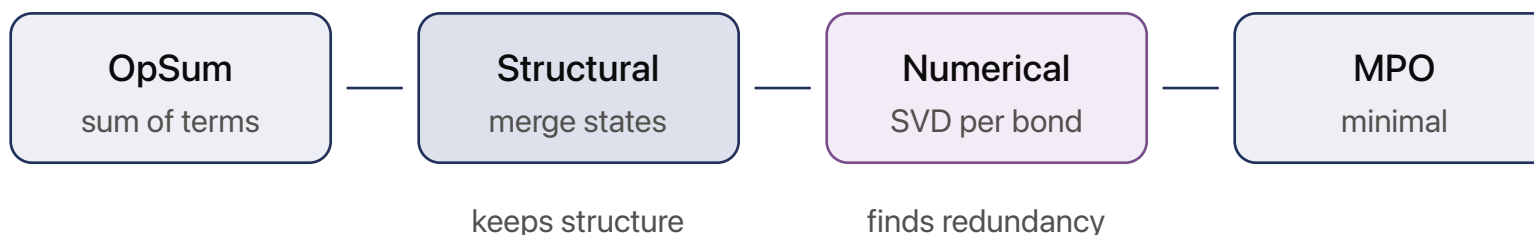
Structural and Numerical Compression

Act on the Automaton:

- Merge equal prefixes and suffixes
- Symbolic compression retains sparsity
- Exact compression
- Cheap enough to run on millions of terms
- Blind to linear dependencies

Act on the Data:

- Rank-revealing QR/SVD
- Destroys sparsity
- Can truncate
- Much more expensive
- Minimal bond dimension



Interface

External Interface

$$H = J \sum_i \vec{S}_i \cdot \vec{S}_{i+1}$$

Simple “sparse” dictionary approach: mapping (unique) operators to their coefficients.

Add syntactic sugar for convenience: +, −, *, /

“Don’t change a winning team!”

```
terms = OpSum()  
for j in 1:(nsite - 1)  
    terms += 1 / 2, "S+", j, "S-", j + 1  
    terms += 1 / 2, "S-", j, "S+", j + 1  
    terms += "Sz", j, "Sz", j + 1  
end  
H = MPO(terms, sites)
```

External Interface

$$H = E_0 + \sum_{pq}^K h_{ij} c_p^\dagger c_q + \sum_{pqrs}^K V_{pqrs} c_p^\dagger c_q^\dagger c_r c_s$$

Larger sums require extra care: at $K = 100$ orbitals, we have $\sim K^4 = 100.000.000$ terms

- Allocating strings can be expensive
 $\implies$ use bits types for operators
- Repeated addition re-copies all terms (K^8)
 $\implies$ use in-place operations

```
terms = OpSum()
for j in 1:(nsite - 1)
    terms += 1 / 2, "S+", j, "S-", j + 1
    terms += 1 / 2, "S-", j, "S+", j + 1
    terms += "Sz", j, "Sz", j + 1
end
H = MPO(terms, sites)
```

External Interface

$$H = E_0 + \sum_{pq}^K h_{ij} c_p^\dagger c_q + \sum_{pqrs}^K V_{pqrs} c_p^\dagger c_q^\dagger c_r c_s$$

Larger sums require extra care: at $K = 100$ orbitals, v

- Allocating strings can be expensive
 $\implies$ use bits types for operators
- Repeated addition re-copies all terms (K^8)
 $\implies$ use in-place operations

```
cd, c = fermion_operators() # bits types

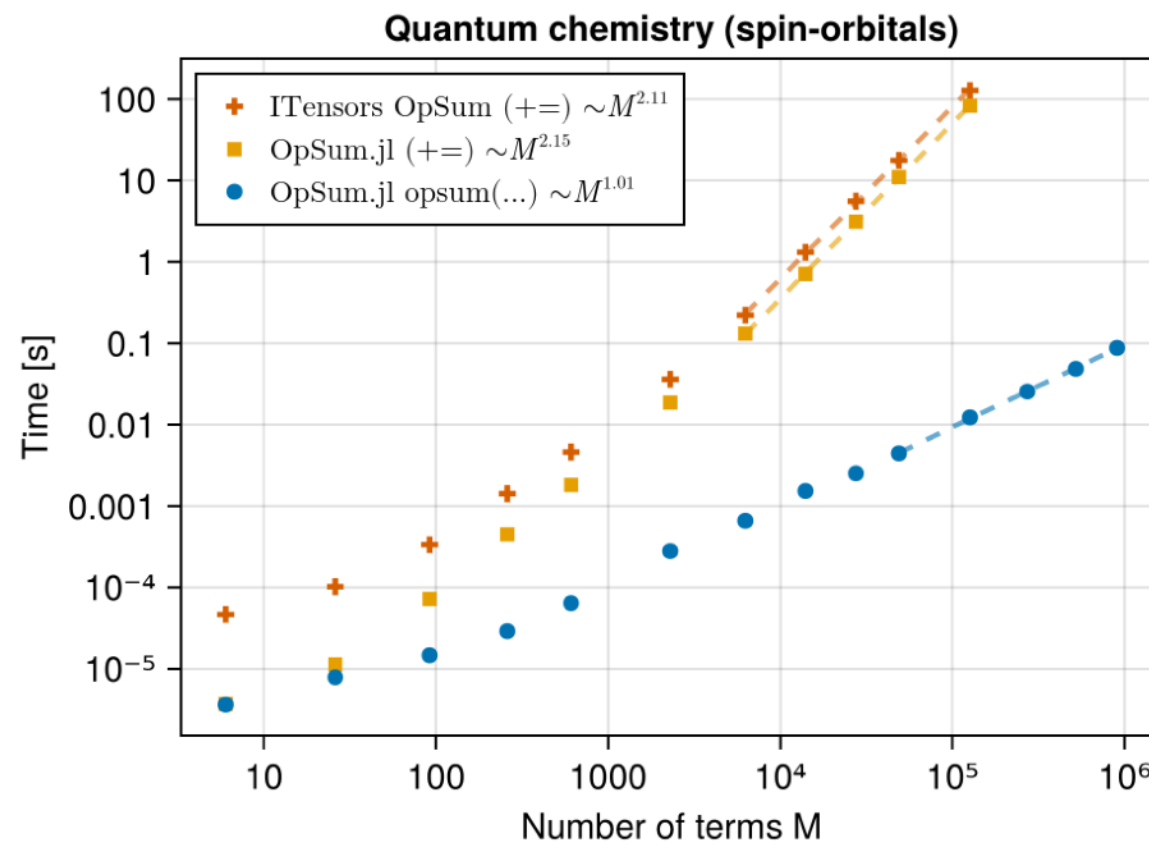
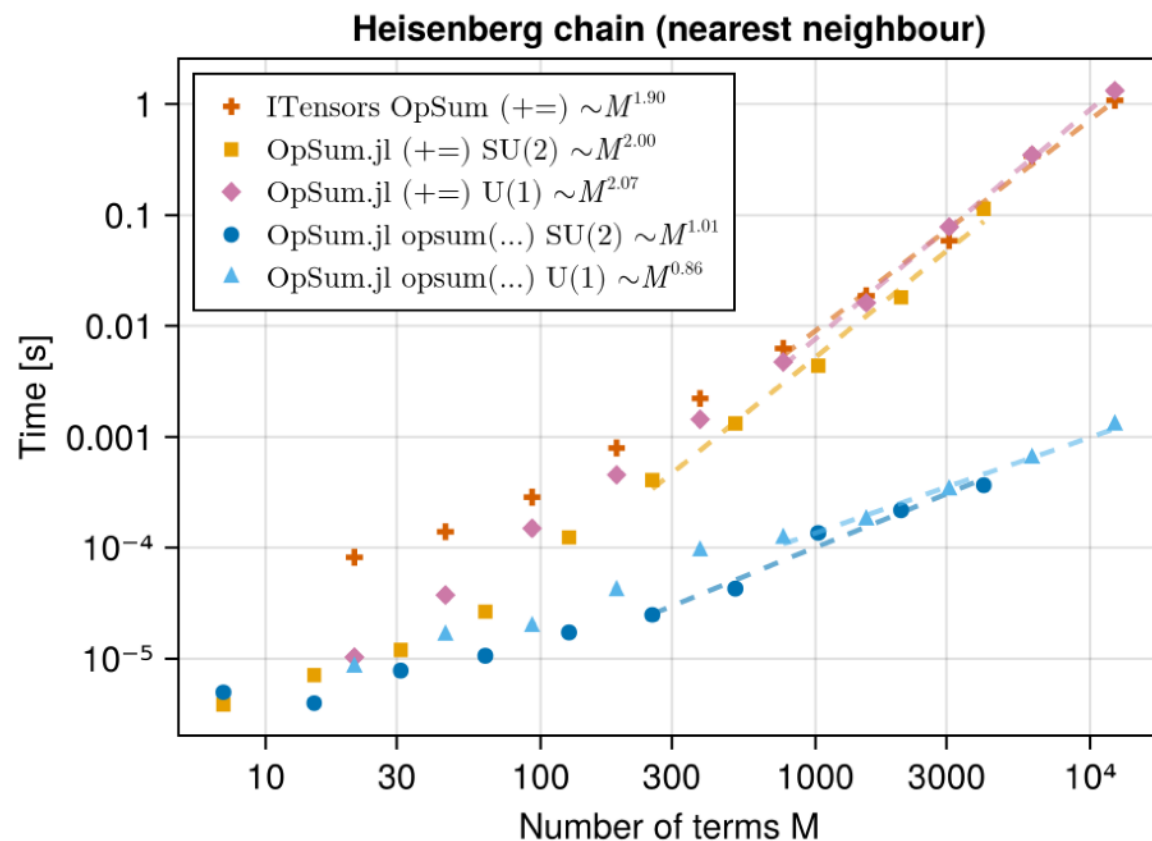
H1 = opsum(
    h[p, q] * cd[p] * c[q]
    for p in 1:K, q in 1:K
)

H2 = opsum(
    V[p, q, r, s] * cd[p] * cd[q] * c[r] * c[s]
    for p in 1:K, q in 1:K, r in 1:K, s in 1:K
)

H1 + H2
```

(Preliminary) Results

Building the operator: += in a loop is $O(M^2)$



Quantum-chemistry model: spinful two-body Hamiltonian restricted to index quadruples with four distinct spin-orbitals; N sites = $2 \times$ orbitals. Power laws are fitted over the largest decade and a half only; the dashed line spans exactly what was fitted. The += curves reach their asymptotic slope only once the copy cost dominates the per-term construction cost.

External Interface

$$H = E_0 + \sum_{pq}^K h_{ij} c_p^\dagger c_q + \sum_{pqrs}^K V_{pqrs} c_p^\dagger c_q^\dagger c_r c_s$$

Additional preprocessing required:

- Simplify repeated operators $p = q$
- Re-order $p < q < r < s$

Requires knowledge about the operator types!

External Interface

$$H = E_0 + \sum_{pq}^K h_{ij} c_p^\dagger c_q + \sum_{pqrs}^K V_{pqrs} c_p^\dagger c_q^\dagger c_r c_s$$

Additional preprocessing required:

- Simplify repeated operators $p = q$
- Re-order $p < q < r < s$

Requires knowledge about the operator types!

```
cd, c = fermion_operators() # bits types

H1 = opsum(
    h[p, q] * cd[p] * c[q]
    for p in 1:K, q in 1:K
)

H2 = opsum(
    V[p, q, r, s] * cd[p] * cd[q] * c[r] * c[s]
    for p in 1:K, q in 1:K, r in 1:K, s in 1:K
)

H1 + H2
```

Extensions

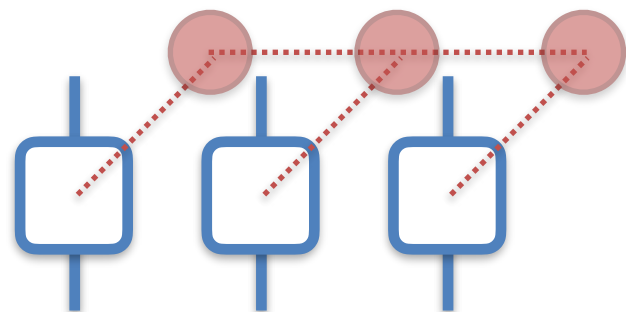
Handling Symmetries

Additional complications arise when handling general symmetries:

- Fermionic anti-commutation relations: $\{c_i^\dagger, c_j^\dagger\} = 0 = \{c_i, c_j\}$ and $\{c_i^\dagger, c_j\} = \delta_{ij}$
- Non-abelian symmetry only allows symmetric combinations: $\vec{S} \cdot \vec{S}$ but not $S_x S_y$

$\implies$ we have to incorporate *fusion trees* to correctly handle the individual operators

See also: [TensorKit.jl](#)



$$\begin{array}{c} c \\ \downarrow \\ \mu \\ \swarrow \searrow \\ a \quad b \end{array} = \sum_v [R_c^{ab}]_\mu^v \begin{array}{c} c \\ \downarrow \\ v \\ \swarrow \searrow \\ b \quad a \end{array}$$

$$\begin{array}{c} d \\ \downarrow \\ v \\ \swarrow \downarrow \searrow \\ e \quad \mu \quad c \\ \downarrow \downarrow \downarrow \\ a \quad b \quad c \end{array} = \sum_{f, \kappa, \lambda} [F_d^{abc}]_{(e\mu\nu)}^{(f\kappa\lambda)} \begin{array}{c} d \\ \downarrow \\ \lambda \\ \swarrow \searrow \downarrow \\ a \quad b \quad c \end{array}$$

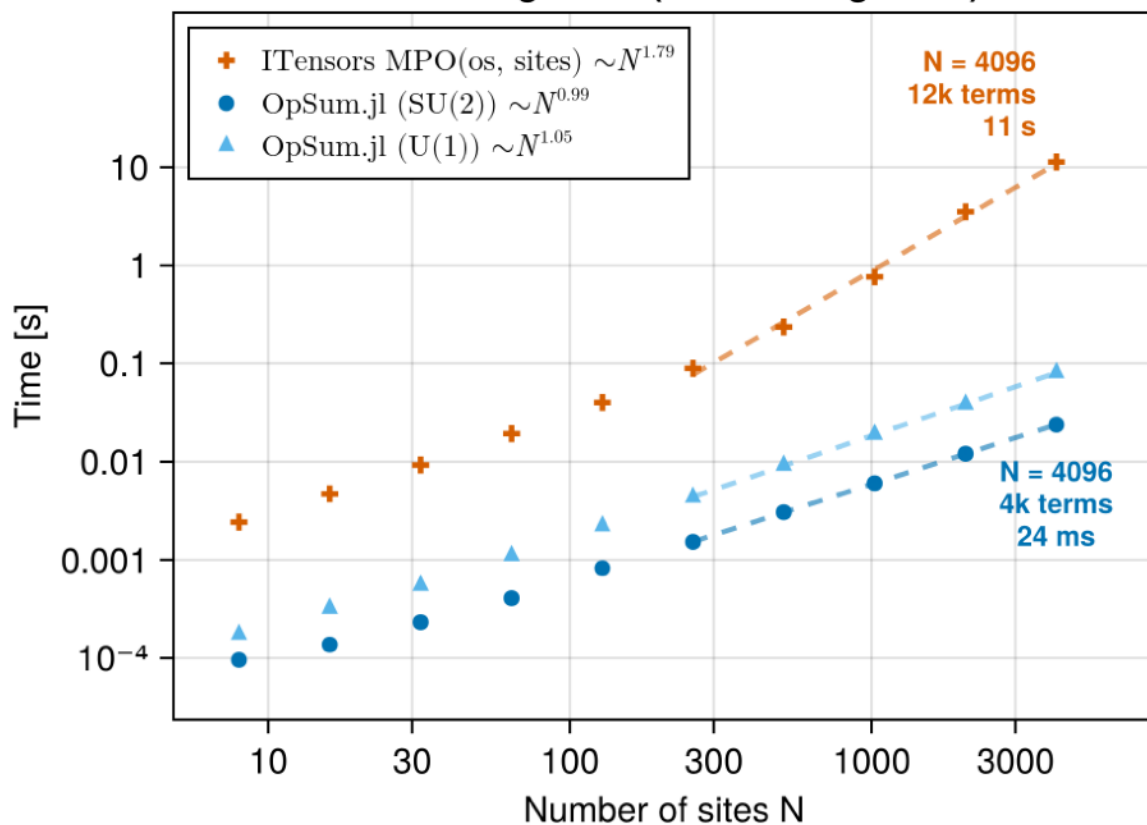
Thermodynamic limit

- Handle infinite number of terms by focusing on a single unit cell
- Compression must be translation-invariant
- Easier: no all-to-all connections?
- Harder: (infinite) long-range connections via sum of exponentials

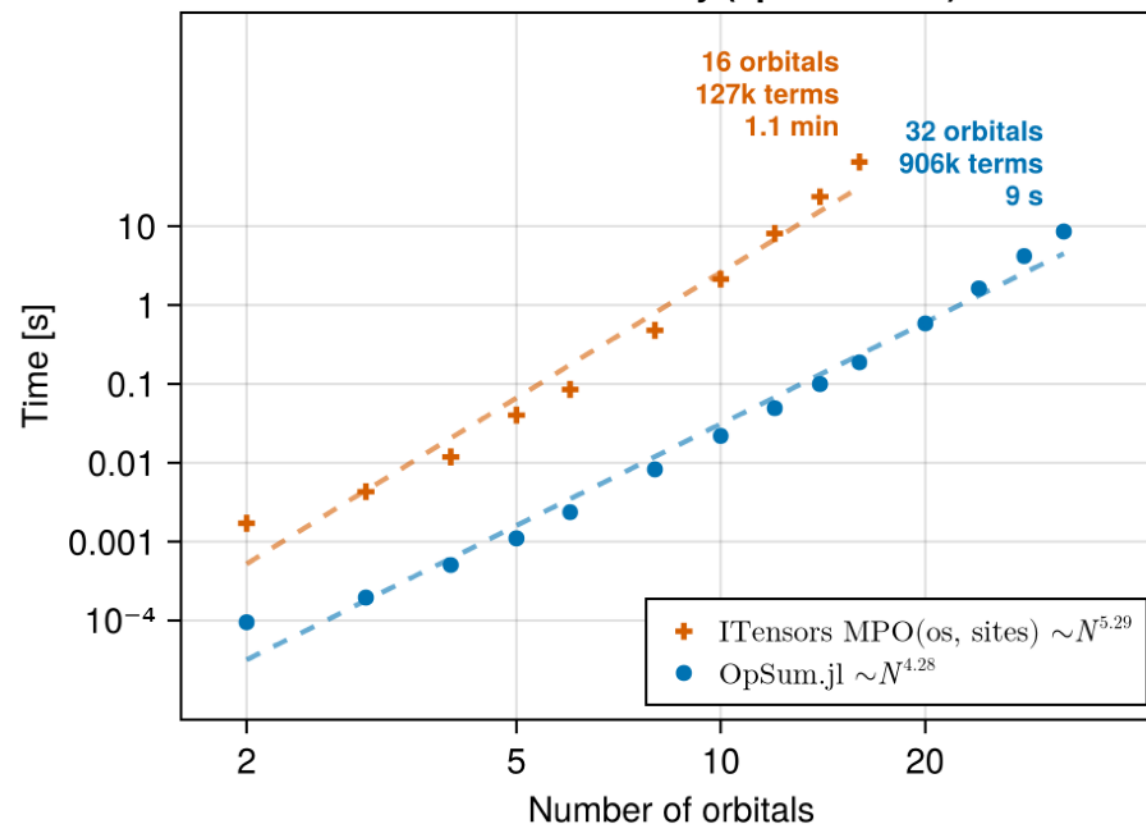
(Preliminary) Results

The whole pipeline: terms $\rightarrow$ MPO

Heisenberg chain (nearest neighbour)



Quantum chemistry (spin-orbitals)



Power laws are fitted over the largest decade and a half only; the dashed line spans exactly what was fitted.

Thank you.

